Making Turing machines useful

Or, how I got Doom to run on a Turing machine

Cayden de Wit*
cayden@4dewits.com

Abstract

Turing machines are interesting[citation needed]. They are, however impractical to do any real physical work with. The infinitely long tape is an impediment on building a true Turing machine in the real world, due to there being a finite amount of matter to construct such a machine out of. And even if an infinitely long tape could be sourced, Turing machines are not user-friendly machines to program[14]. Being made more for mathematical proofs and not real world programming makes them terse to work with. However, there are solutions to these very serious problems. Despite being able to run any well described algorithm on the natural numbers (See the Church-Turing thesis), Turing machines are not used to do so. And as an ardent believer in the idea that everything should be practical, I believe that it is pertinent to demonstrate the efficacy of this marvelous property of the Turing machine with an impractical example. In this paper, I will finally allow Turing machines to run any program I choose and use this new machine to run some most useful and entertaining computations.

1 Disclaimers

***Disclaimer**: There is going to be very little mathematical rigour in this paper. It might also cause minor discomfort in the reader. Reader discretion is advised.*

I also apologise if this paper sucks. It is my first and the tone might slightly non-academic in nature but my will to strip out all instances of narrative and first person pronouns have decline exponentially over the time it has taken to write this paper.

2 An overview of the methods

Turing machines are capable of running any algorithm (on the natural numbers) that is effectively calculable. This means that so long as one can construct a Turing machine to compute what you want, it will do so (duh). The issue comes with constructing the Turing machine. They are unintuitive to work with and doing any real computation with them in a general way is hard to do.

The goal of this paper will then be to simulate a real computer inside the Turing machine: Expediting the hard work away to working with more familiar ways of doing computation (Such as the mathematical "Random-Access Stored Program machine" which could allow the programmer to program in more familiar assembly).

The reason this works is because of a fact about Turing complete systems: They can *in theory* simulate each other (As a product of using bisimulation to prove equivalence of two machines). Some models might be better at computing different tasks/be more efficient/be simpler to describe, but they are all theoretically equivalent.

So, in the same way that a Turing machine can simulate itself (i.e: The universal Turing machine), one can simulate some other computer inside a Turing machine. This will be the methodology of running arbitrary programs on the Turing machine with (relative) ease.

3 Programs

Whilst having a computer able to run instructions is all well and good, computers are best used when they have a use and the best use of a computer is entertainment. Whilst some have cited the NES as the computer delivering the maximum entertainment value[10], my belief is that a possible contender for that position is any computer capable of running Doom.

Doom needs no introduction, but I will give it one anyway. Doom is extremely light, built to run on hardware that doesn't have the same power as modern computers today[citation needed]. The Doom source code is also very portable, being written in quite minimalistic and standard library light C. It is possible that for these reasons (and possibly due to a meme caused by the previous facts), Doom is widely ported to all sorts of interesting hardware platforms (r/itrunsdoom for reference). Doom is also objectively fun. All these facts come together to form the perfect program to try and run on a Turing machine.

*Pronounced [keidən də vət]

[1] `https://github.com/ozkl/doomgeneric`

I took the popular Doom source port doomgeneric[1] and was able to completely free it of the standard library, only requiring a minimum of about 11 functions to run it. This makes Doom an ideal program to port: No need for patching a standard library together.

Thus, the goal of the Turing machine constructed will thus be to have the ability to run Doom, in some capacity.

Figure 1: hell yeah

4 The mathematical machine

If one wanted to be extra rigorous in their terminology, a Turing machine could never exist. With the tape being finitely long in the real world, it technically prevents it from being a Turing machine. Rather, it should be classed as a Linear Bounded Automaton: a Turing machine with a bounded tape (Depending on the definition of such a machine and the input, the tape could be any finite length). This model more closely resembles a real life computer with it's finitely long memory.

Interestingly enough, this does mean that computers as we have them are not *technically* Turing complete due to them having finite memory. This detail doesn't particularly hinder our ideas, however. The problems that we want to solve on a computer don't require unbounded memory. Thus, I will not attempt to define a system that can use unbounded memory as I am, at the end of the day, interested in running real world code on a "Turing machine" so the unbounded tape is not required.

I will from hereon out refer to the mathematical machine of this paper as a "Turing machine" but it is, in theory, not a Turing machine[2] as it will be implemented in software as having a finitely long tape that we will assume will be "good enough" for the problem.

Without further ado, the definition of the machine will be adapted from the definition given by Cohen[3]. I am not going to define this machine with extreme rigour (This is left as an exercise to the reader) because it isn't particularly important for this paper. However, to highlight the important details:

- The tape alphabet (Commonly denoted as Γ) and the input alphabet (Denoted Σ) will be the set $\{\Delta, 0, 1, O, I, A, S, C, V, P, F, R, M, *, \#\}$. Where Δ indicates the blank symbol.
- The transition function need only to shift the tape left or right (In other words, no "no shift" direction).

Now that we understand roughly what we are expecting out of our mathematical model, let's turn our attention to the real world hardware we wish to simulate on this machine.

5 RISC-V

RISC-V is an open source Reduced Instruction Set architecture. It is gaining traction in the embedded programming where it is perhaps shaping up to gain real mainstream adoption[6]. It is also surprisingly simple. Depending on what laundry list of extensions you tack onto the base instruction set, one can choose between having a drop-dead simple (yet usable) architecture capable of doing something all computers can do in 38[†] instructions (Whilst not feeling like programming in brainf*ck or subleq) to processors that can potentially be used in laptops with all the needed bells and whistles to make it a smooth experience.

This paper is primarily concerned with computation heresy, so the simple architecture is going to be the focus of this paper. The specific extension list is RV32I, which means the base instructions and 32 bit word width. This architecture offers 38 instructions and with those 38 instructions, one can run pretty much anything desirable (If not a little slowly, what with not having any floating point hardware, vectorised instructions, hardware multiplication, etc.). This extremely simple architecture is so simple, one can implement it in about 250 lines of tightly woven C. This extreme simplicity lends itself well to tomfoolery and abject silliness.

Since the "algorithm" of running RISC-V code is so simple to describe, it *should* be humanly possible to construct a Turing machine to run said algorithm and therefore, be able to run anything a RV32I CPU can run. Thus, this is the goal: Write a RV32I emulator Turing machine and then run Doom on that.

6 Side effects may include

There is one small problem, however. Being able to compute any algorithm does not constitute all of a modern computer's functions. Often time, we wish to have our computer model a side effect. Or, we wish to get some sort of "input" that can't be deduced from the initial state because the input itself is not computable such as a pure random number or a keyboard input from a user.

[2] Can you clickbait someone into reading a paper? Readbait?

[†] Those instructions are: `lui`, `auipc`, `jal`, `jalr`, `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`, `lb`, `lh`, `lw`, `lbu`, `lwu`, `sb`, `sh`, `sw`, `addi`, `slli`, `slti`, `sltiu`, `xori`, `srli`, `srai`, `ori`, `andi`, `add`, `sub`, `sll`, `slt`, `sltu`, `xor`, `srl`, `sra`, `or`, `and`, `ecall`. 38 total. It's 37 without the extra `ecall` instruction which we need to handle side effects. I am also not including `ebreak`.

This is not the first time this sort of problem has been encountered though. Other mathematical ideas that gave been turned into machines/programming languages have found ways to model both these things. Haskell famously does this using "monoids in the category of endofunctors"[8].

However, unlike the λ-calculus, the "objects" of a Turing machine are not comprised of functions. Rather, they are the nodes and tape. The solution would be to have some or other "node" that uses the tape as an instance of a problem to solve which we can't solve ourselves (Such as gaining keyboard input from a user, which would be quite extraordinary if we could compute that in a Turing machine) and alter the tape to the desired answer (And perhaps do some side effects on the side, like printing to the screen).

Enter the Oracle machine: A mathematical machine used by smarter people than me to do some real mathematics. By attaching an "oracle" to our Turing machine, we may enter a node that may query the oracle for information that the Turing machine is incapable of computing/perform some outside action cleanly. With that, we amend the definition of our machine slightly, but have a way to model all the IO we could ever need.

This is an idea that has been around for a while (Although, under a different name). Alan Turing himself thought of so-called "choice machines". These machines had states that would wait for "some arbitrary choice... by an external operator"[13] when reaching a state that required some incalculable result. In our case, the "external operator" is simply an oracle that uses the tape as an input to a problem.

7 Software

Now that we have an idea of what we're doing, I need some software to construct the Turing machine in and run it on. Whilst some very good software exists to run and design Turing machines, they do not fit my use cases. The editors I found are cumbersome and perhaps too refined to use quickly. The emulators they come with are also not designed to run at maximum speed/give the debug output I need. Since the designed machine is likely to be a few hundred states large, I need to be able to run it quickly and have an easy-to-use editor.

Contributing to editor wars even further, I took it upon myself to write a (bad) Vi like Turing machine automaton editor. I also constructed a (bad) standard format for storing Turing machines on the disk and a compiled version to be loaded by a Turing machine emulator to run the machines as fast as possible. The emulator preloads and preprocesses the compiled format, so no time is wasted in running, only the logic of a Turing machine is executed.

[3] `https://github.com/adamyedidia/parsimony`
[4] `https://github.com/sorear/metamath-turing-machines`

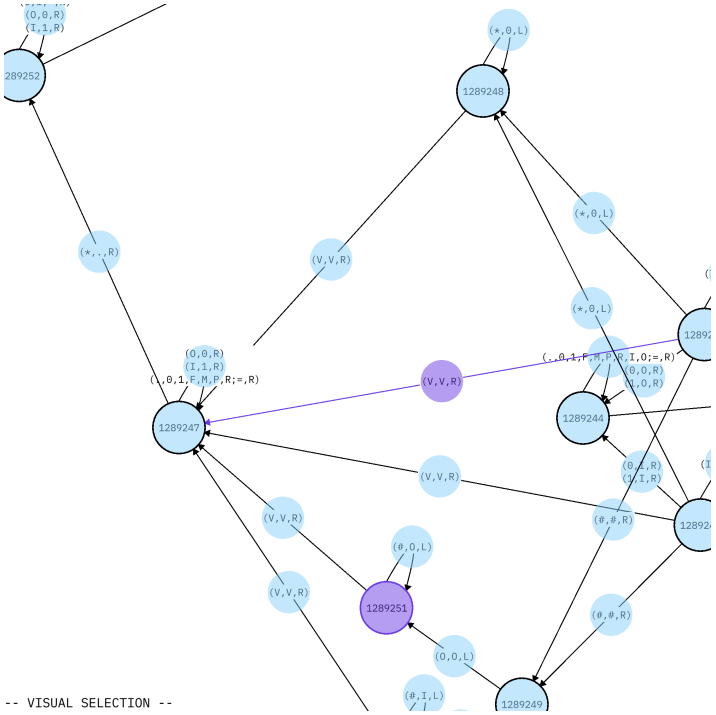


Figure 2: A screenshot of my horrible editor. It does work though and that's all that matters.

One this I did find after having already completing the Turing machine are Turing machine languages, such as Laconic[3] or (The one that seems more plausible for me to use) Not-Quite-Laconic (NQL)[4]. The curious reader is free to try and adapt these tools to be able to construct the RISC-V Turing machine, but that is beyond the scope of my patience.

8 Methodology

8.1 Tape layout

The first decision to make is how to represent data. One of the more common ways[citation needed] to store numeric data on a Turing machine is to use unary, but this does not make storage or operations on the data very easy. The most obvious way would be to store it in binary. Binary has the advantage of being used by computers already and having very simple arithmetic, which helps when working on a Turing machine.

The next problem to solve is what to store on the tape and where. A good place to start would be to have a minimal reference implementation of the RV32I instruction set in code to attempt to at least find out what needed to be stored. After writing the code, one can start using the code to follow what objects are needed in certain places.

I decided that every variable used in the code should get its own spot on the tape. Essentially rewriting the entire C algorithm piece for piece in the Turing machine.

For example, the main structure of the machine includes the following members:

```
typedef struct riscv {
  uint8_t *mem;
  uint32_t pc;
  uint32_t regs[32];
  int flags;
} riscv_t;
```

Listing 1: Main structure of the C emulator

There needed to be some way of keeping track of the current execution position (`pc`), 32 registers (one of which always has to be zero), flags and some indeterminate amount of memory.

I would also need space for variables outside the main structure. I wrote the code to use four temporary variables at a maximum and added space for those variables. There is also a space for the fetched opcode, a calculation block, return value and return register.

After all possible needed values were taken account of, the final version of the tape looked like this:

A	Opcode						
S	S_1	Δ	S_2	Δ	S_3	Δ	S_4
C	Calculation						
V	Store val	Δ	Store reg				
P	`pc`						
F	Flags						
R	Bounce	Δ	R_0	Δ	$\cdots$	Δ	R_{31}
M	Bounce	Δ	M_0	Δ	M_1	Δ	$\cdots$

The newlines are just to provide visual clarity and in reality, these all sit on 1 straight tape, not multiple individual tapes. The wider blocks are comprised of 32 symbols each (32 bits) and memory is divided into 8 symbol chunks. Every section is demarcated with a unique symbol. This makes it easy to navigate to whatever section is required for the current computation. Within sections, there might have been multiple "values", these were separated with the blank symbol.

The sections are used as follows: The A section is used to hold the current opcode, so the machine doesn't have to seek into memory every time some information is needed from the opcode. The S section is used to hold temporary variables and is used as internal storage. The C section is used to store any finished calculation results. The V section consists of a place to store the value that will be stored in the return register and the return register number. The P section is used to store the current program counter. The F section is used to store any machine flags. The R section is used to store the registers and has a bounce register in front of the 32 registers. The M section is the main memory and has a bounce register in front.

In the end, I got lazy and didn't use the C and F sections. I was able to shoehorn any calculations into the variable section (S) and I didn't need any flags for the machine to work. I could have removed them, but that would take effort and fair bit of rewriting since sometimes it's easier to get to the end of a section by using another section's marker as a marker. So, whilst the contents of the F section were never used, the F marker was very useful.

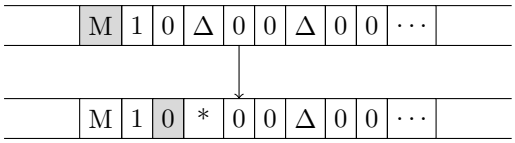
The main data of the tape will be stored in binary using the 1 and 0 symbols. Some "alternative" symbols (Useful when copying but without erasing the data that is already there) for the binary symbols will be I and O (Since they look like the first two Arabic numerals). Lastly, $*$ and # will be used as general purpose markers.

8.2 "Array" access

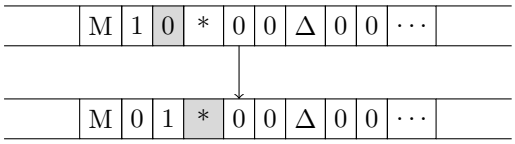
The main problem to solve was how to handle what in the C code was programmed as arrays: Accessing registers and memory. Whilst it was perhaps possible to keep each individual register as a separate variable and give each one its own unique symbol and go to whatever one was needed, the same could not be said about memory.

The solution to this problem was to include those extra "values" in front of the register and memory sections. These would act as "bouncers" that would allow arbitrary access to whatever address of memory (Or, whatever register) was needed. The algorithm works as follows (Note: 2 bit numbers are used as an example. Not on the actual machine):

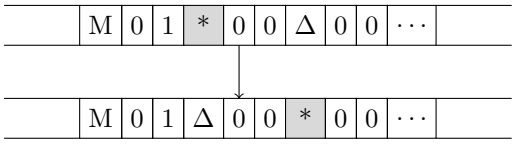
Step 1: Go to the end of the counter and add a star (Or, any distinct marker) to the first space:



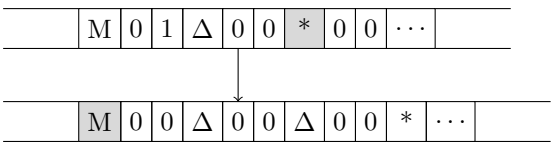
Step 2: Subtract one from the counter and move right until the marker is found:



Step 3: Replace the marker with a blank, move over the current cell and replace the end of it with a marker:



Step 4: Move back to the counter and repeat the subtraction, find marker, shift marker right manoeuvre until the counter reaches zero:



What this does is push a marker further out onto the tape, one value at a time until the counter (Bounce) hits zero. This has the effect of putting the marker at the beginning of the nth memory cell (Or, register cell). The reason I call it a

"bounce" is due to the reciprocating motion of the tape head, going out, coming back and going out again.

The attentive reader might notice that this algorithm takes longer the larger the memory address is. In fact, since the machine has to go to the address, then return, then go out again and then go further and repeat, memory access is actually $O(n^2)$ with n being the size of the memory address. This likely won't be a problem in any way going forth[5].

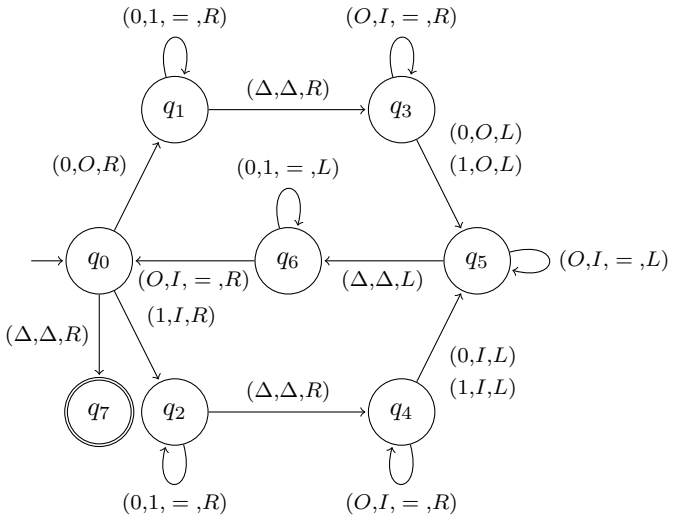
8.3 Numbers

A since we plan to represent numbers as binary, we will have to be able to do mathematics to them, such as binary addition or subtraction. Addition is fairly easy and can be done in the usual way we are taught in school, start from the least significant digit of the first number and add it to the least significant digit of the second number. Carry (if needed) over the rest of the number until we don't need to anymore. The only thing of note is to stop carrying if we overflow. Subtraction is also fairly easy. Since RISC-V uses two's compliment, subtracting is as easy as negating the second number and adding it to the first.

8.4 Copying

A lot of the machine's job is going to be copying data from one cell to another. The simplest way I found to do this was to start at either end of the number to be copied, get the digit, replace it with a placeholder, move to the appropriate place and put the digit down.

The logic looks something like this for a Turing machine to copy from the front of a number into a number 1 blank symbol over to the right of it:



This will leave the tape covered in temporary storage symbols: O and I. These will have to be cleaned up by replacing them with 0 and 1 respectively and then execution can continue as normal.

There is one extra consideration: Endianness. Storing numbers in registers and temporary variables should be in big endian for easy of computation. However, little endian memory is needed for memory. Memory is already broken up into 8 bit chunks, which is convenient. However, fetching and storing values to/from memory will require an endian change. This is not too difficult to do, one just has to remember to do it in the copy when accessing memory.

8.5 Calls

As discussed previously, there are certain operations the Turing machine cannot perform itself. But what operations do we actually need to give to the oracle to do? My custom Doom source port tries to be as minimal as possible in terms of required calls. The gist of what we need is: A way to open a window for drawing, a way to draw some data to the window, A way to delay the processor (sleep), a way to get some sort of "tick" value (for timing), keyboard input, a way to print characters to the screen (for debugging), file opening and file reading (Needs the Unix calls of `open`, `read`, `tell` and `lseek`).

Keyboard input and timing will not be needed due to the slow nature of the machine. However, I added calls for them just for fun.

8.6 The machine cycle

Now that we have a few components, we can put them together into the machine cycle for the Turing machine. It looks something like this:

- Fetch
  - Copy `pc` into the memory bounce area.
  - Perform the memory bounce.
  - Copy the contents of that memory address into the area for the opcode (Remember, little endian copy).
- Decode
  - Navigate to the opcode section of the instruction and figure out what instruct we need to execute.
  - Every individual instruction can decode additional data into the temporary storage areas or bounces.
- Execute
  - Do whatever execution is required of the instruction.
  - Store the return value in the return value part of the tape.
- Store
  - Copy the return register into the register bounce.
  - Perform the register bounce
  - Store the return value in the required register (If it is not the zero register).
  - Add four to `pc`.

[5]Foreshadowing is a literary device in whi–

And repeat ad infinitum.

9 Results

You've made it through all the boring theory. Now, we can finally evaluate the results.

Putting the machine together in its entirety took about 2 weeks. The machine consists of exactly 1337 nodes and 2293 arrows between the nodes (Each arrow can hold multiple transitions). It is comprised out of 4765 5-tuples (transitions). This makes the machine larger than the hypothetical smallest machine needed to prove ZFC inconsistent[1]. It also looks hideous, it can't even be nicely printed on one page: LaTeX fought with me every step of the way trying to include the image (See Figure 3).

9.1 Inner workings

The final Turing machine consists of about 12 different parts that work together to make the final machine. Some of the parts do more than others but the grand laundry lists of parts contain:

Instruction acquisition: Load `pc` into the memory bounce, bounce to the right memory address and copy the 32 bit instruction little endian-wise into the A section of the tape.

Instruction decoding: Store the return register in the second part of the V section and return to A. Navigate to the end of the instruction and read the 7-bit opcode. Depending on the bits read, transition over to the correct section next.

ALU: Perform some sort of arithmetic/bitwise operations on the values stored in the S section of the tape. These values will be copied here by different bits of machinery depending on whether the opcode called for an immediate operation (Opcode `0010011`) or double register operation (Opcode `0110011`). Once the operation is done, store the result in the V section.

Load upper immediate: Loads an upper immediate as per the RISC-V instruction (`lui`).

Add upper immediate to program counter: Fetch `pc`, add some immediate to it and store as per the RISC-V instruction (`auipc`).

Unconditional jumps: Perform to an unconditional jump to the address stored in S_1 after storing $pc + 4$. The value jumped to depends on whether the instruction was a register jump (`jalr`, opcode: `1100111`) or an immediate jump (`jal`, opcode: `1101111`).

Condition jump (branch): Compare values stored in the S section and jump to a location if some condition holds (Depending on the instruction).

Load memory: Load a value from a memory address (After performing memory bounce, etc.), copying the value in little endian-wise. Make sure to only fetch as many bytes as requested and pad to the desired length.

Store memory: Store a value from a register into memory. Perform memory bounce and copy value into memory. Only copy as much as needed.

System: Used in `ecall`. Make a call to the oracle.

Store register value: Most opcodes have a result to store in a register when they finish executing. If the store register is not 0 (RISC-V register `x0`), then do a register bounce and copy the value of the first part of the V section into the appropriate register. Instructions that do not return values to registers can skip this part of the machine.

Increment program counter: Add four to the program counter. Jumps may skip this section since they set `pc` and do not have to be changed.

9.2 Benchmarks

The next logical step of having the machine would be to benchmark it, this way we can compare it to other great silicon and software RISC-V machines. The one small issue is that it is hard to get a feeling of how long the average instruction takes to go through the machine cycle (Since fetching the instruction is $O(n^2)$, the time is different depending on where the instruction is). However, facts like this have never deterred me before.

Writing a quick C program, compiling it and running it lead to an average instruction execution time of about 0.2664s per instruction. Or, about 3.7535 instructions per second.

To compare this to other RISC-V processors (Assuming the worst: That *every* instruction takes 32 clock cycles to complete which is likely not true) and the C emulator:

RISC-V Processor	Instructions per second
RP2350's Hazard3 processor[12]	4.69×10^6
ESP32-C3 RISC-V processor[5]	5×10^6
GD32VF103CBT6 (Processor used in the Longan Nano)[7]	3.38×10^6
Milk-V Duo RISC-V CPU[9]	32.25×10^6
C emulator	12.69×10^6
Turing machine RV32I	3.75

Table 1: oh no

To further add to this disaster, comparing it even to ancient vacuum tube computers still disappoints. Even the Manchester Small Scale Experimental Computer could do better with about 830 instructions per second[11]. It beats only the Z4 which processed about 0.278 instructions per second[11], but even this victory is short-lived: If the memory addresses were raised, it would start running far slower.

Figure 3: This consumed two weeks of my life and I feel unspeakable wrath and rage when it graces my computer screen. This is compressed to high hell due so that it may fit on the page and so that the full 5.2MiB image isn't loaded every time the document is opened. So, the curious amongst you can find the full digital image here: `https://bitbucket.org/caydendw/turing-computer/raw/fdf9c60664f8cfde9d57afc6644fe2380e81dcfc/whole_machine.png`. As a further curiosity, the attentive reader might notice that I was quite thorough in documenting the machine by labeling the nodes until I got sick of doing that. You'll also notice that the node numbers are absurdly large in some places. This is due to me being lazy in how I assigned node IDs in my program to edit these machines, specifically how I dealt with copy and pastes.

10 Damn, Turing kinda got hands

Writing some stub code to analyse where the bottlenecks are revealed the main points of contention: Memory bounces (As was suspected). The biggest surprise is how large the bottlenecks were: With the `load` instructions passing through one node 15553471881 times in 109 load instructions (About 142692402 times per load). Every node with a large visit count is one that performs some or other memory bounce. It is very evident that is $O(n^2)$ complexity memory access is the cause of the slowness of the Turing machine.

So, the machine is slow, and it is the memory's fault. The first question to ask is: Skill issue? Well, after doing some research, it turns out that it is not. To use a result proved by Cook and Reckhow[4], I believe that it demonstrates that I am (and this is the scientific term for this) poked. I am doomed to use quadratically more steps on the Turing machine and since all of my other algorithms run in constant (Addition of 2 fixed width numbers, loading upper immediate, etc.) or linear (Copying numbers from memory to registers) time, $O(n^2)$ random access is the best we can do.

So, we are in a position where memory access is pretty much given to be $O(n^2)$ no matter what I do, meaning that the only hope I have left is to speed up the machine to a desirable level, or cheat. Speeding up the computations seems *slightly* unlikely. Some more analysis lead to me finding that every transition from one node to another takes approximately 22.34ns on my computer. This is already quite fast and even speeding it up to 1ns would only scale up the speed by a factor of 22, not enough to run Doom. Running it any faster than that is also infeasible[6]. Attempting to parallelise it seems hard to do without some form of branch prediction which I am not smart enough to do.

[6]There is one option to consider: The linear speedup theorem. This does theoretically do what I need, however I struggle to imagine how I would do that given my badly home-brewed software, and I'm not redoing that Turing machine.

This leaves me with very little choice but to face the music, and cheat.

11 Cheating

So, how is the cheating done? The easy answer is just to just put the marker down on a memory bounce when we come across a memory bounce of some sort: Doing the Turing machine's work in software for it. This doesn't change how the Turing machine works or what it does, it just makes it slightly faster to simulate. If one were so inclined, they could run the machine without the cheats, and it would produce the same results (Albeit, perhaps a few weeks later). Doing this is relatively simple as only 3 memory bounces are done in the machine: For fetching the current instruction, `load`s and `store`s. Adding some code to the Turing machine interpreter to do this brings the Turing machine up to a blazing 44.2150 instructions per second: Not good enough (Even though it is almost a 12 times increase in speed).

Reanalysing the bottlenecks show that long distance copies between memory and the start of the machine are slowing it down even more. This is not a part of the $O(n^2)$ memory bounce issue, it is more to do with the $O(n)$ (Where n represents the memory address) algorithm of copying a value from memory to somewhere else. This could be fixed by just setting the tape head to where it needs to be. Often times, the tape head needs to get from far out in memory to a known place on the tape (Say, the S section). This can be sped up by simply setting the position of the tape head to the where the S section begins.

All of this is does not change the behaviour of the Turing machine, it all it does it do what the Turing machine was already going to do, a little bit faster. If one were so inclined, they could remove all the cheats and wait a month for Doom to run without the cheats, but I do not have the patience.

I added 85 shortcuts to the machine. Most of these were jumps to locations that were far away and could be predermined (For example, moving from far out in memory all the way to the V section). Moving from far away to another place could take thousands of loops on 1 node and if the location is easily determined, a lot of processing time can be saved. Of course, the $O(n^2)$ memory accesses was also "cleaned up" so that it ran a lot faster.

Fixing a lot of large bottlenecks brought the machine up to a blistering 288.2828 instructions per second. I decided that this was a good enough speed to try running Doom.

And slowly it did run.

12 Analysis: The revival

"Running" Doom might've been a slight stretch of a claim. Whilst Doom did in fact "run", it took 20.4028 hours of continuous execution to get a simple opening title. Furthermore, it took a little less than 8 hours to render a single frame of output (7.8889 hours exactly) and that's not counting *actually* running the game, just the time it took to render the title screen and menu and if my assumption is correct, it would take far longer to render even one frame of E1M1 gameplay.

Caveats aside, it is still running Doom and displaying the results. It might not be playable, but I do consider it a success. To do another comparison, I believe it is useful to compare this result to the results of other projects, useful and impractical (All measurements taken on the same computer unless cited):

Game system	Frames per second
RISC-V C emulator running Doom	31.5315
Native x86_64	717.5780
Crispy Doom without VSYNC and high resolution	543.3463
The Dr. Tom Murphy VII hfluint8 NES emulator[10][7]	0.1154
The Turing machine RISC-V computer	3.5211×10^{-5}

Table 2: Comparison of results

Converting the frames per second to more friendly units, it comes out to 28399.8885 seconds per frame. Or, about 7.8 hours to render one frame. But despite the absolute waste of computing time required to run such a program: I did get the sought after screenshot:

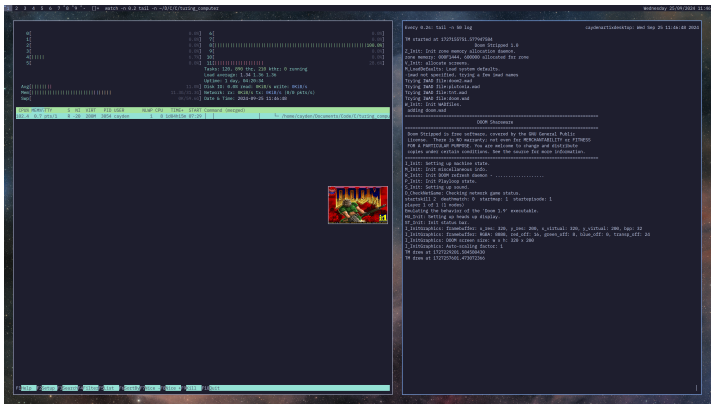


Figure 4: The final result of running the Turing machine for 28 hours, 17 minutes, 29 seconds, 895 milliseconds, 124 microseconds and 673 nanoseconds but who's counting right?

Needless to say, this is completely unplayable. I don't have the patience to wait for Doom to load a single frame of E1M1.

13 Conclusion

The phrase "Turing complete" is a little of catch-all phrase among programmers[citation needed]. The Church-Turing thesis assures that a Turing machine should be able to do what any language does but up until now, I don't think I have seen any-

[7] This *was* for the NES running NES games but just go with the flow

one attempt to put this theory into practice and actually try to run a normal, real world, Turing complete programming language on the titular Turing machine. I think that I have demonstrated the computational power of a Turing machine effectively.

This does of course mean that one can now *Theoretically* run anything on a Turing machine. If one were so inclined, they could implement further extensions of the RISC-V specifications on the Turing machine with MMIO. This could allow them even to run Linux on the machine. It's not too far-fetched either. Just having atomics, Zifencei and Zicsr should be enough to run Linux[2].

However, this silly idea taken far too seriously has come to an end. I wish to thank the readers of this paper for bearing with the absolute horror that has unfolded in these past n pages.

13.1 Evaluation of results

Considering the goal of running Doom on a Turing machine, I would consider this endeavour to be a success. However, there is probably some work still to be done before the machine is used in wide mainstream production. However, I believe the potential is there.

13.2 Similar work

This constructed machine *almost* forms the basis of a proof of the equality of the Linear Bounded Automaton and the word RAM machine with limited registers. Someone with a better sense of mathematics than myself should be able to change this idea (Whilst making it no longer RISC-V) to run a real word RAM machine with limited registers and thereby construct a proof that the two machines are equivalent, but I leave that to other people.

To illustrate this point, this construction was (to my surprise) vaguely similar to that provided in Cook and Reckhow's article about the equivalence of the RAM machine and the Turing machine[4]. Specifically, how to construct a RAM machine inside a Turing machine.

13.3 Future work

Absolutely not.

References

[1] Scott Aaronson. *New comment policy*. 2024. URL: https://scottaaronson.blog/?p=8131 (visited on 11/09/2024).

[2] cnlohr. *mini-rv32ima*. https://github.com/cnlohr/mini-rv32ima. GitHub repository. 10/07/2024. (Visited on 26/09/2024). Commit: 997abfcae9df854fdc62ebdfe66663f6db2bc7d5.

[3] Daniel I.A. Cohen. *Introduction to computer theory*. 2nd ed. New York: Wiley, 1986, p. 553. ISBN: 978-0-471-80271-6.

[4] Stephen A. Cook and Robert A. Reckhow. 'Time bounded random access machines'. In: *Journal of Computer and System Sciences* 7.4 (1973), p. 362. ISSN: 0022-0000. DOI: https://doi.org/10.1016/S0022-0000(73)80029-7. URL: http://www.cs.utoronto.ca/~sacook/homepage/rams.pdf.

[5] *ESP32-C3 Series*. Version 1.8. Espressif. 2024. URL: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf.

[6] RISC-V Foundation. *Semico Forecasts Strong Growth for RISC-V*. 2019. URL: https://riscv.org/riscv-news/2019/11/9679/ (visited on 13/01/2025).

[7] *GD32VF103CBT6*. 2023. URL: https://www.gigadevice.com/product/mcu/mcus-product-selector/gd32vf103cbt6.html (visited on 11/09/2024).

[8] James Iry. *A Brief, Incomplete, and Mostly Wrong History of Programming Languages*. 2009. URL: http://james-iry.blogspot.com/2009/05/brief-incomplete-and-mostly-wrong.html (visited on 25/09/2024).

[9] *Milk-V Duo*. 2024. URL: https://milkv.io/docs/duo/overview (visited on 11/09/2024).

[10] Tom Murphy VII. 'GradIEEEnt half decent'. In: *In Proceedings of SIGBOVIK 0x2023* (2023). Association for Computational Heresy, pp. 52, 54. ISSN: 2155-0166. URL: http://sigbovik.org/2023/proceedings.pdf.

[11] Gerard O'Regan. *Early Commercial Computers and the Invention of the Transistor*. Cham: Springer International Publishing, 2021, pp. 122, 133. ISBN: 978-3-030-66599-9. DOI: 10.1007/978-3-030-66599-9_6. URL: https://doi.org/10.1007/978-3-030-66599-9_6.

[12] *RP2350 Datasheet*. Raspberry Pi. 2024. URL: https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf.

[13] Alan M. Turing. 'On Computable Numbers, with an Application to the Entscheidungsproblem.' In: *Journal of Symbolic Logic* 2 (1937), p. 232. DOI: 10.2307/2268810.

[14] Cayden de Wit. Personal Experience. 07/09/2024.